2do Parcial de Arquitectura y Organización del Computador - 2c2025

Normas generales

- El parcial es INDIVIDUAL
- El parcial es a libro abierto, pueden disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada o a cualquier otro recurso que consideran. Sin embargo, **está totalmente prohibido el uso del celular, internet y chatbots.**
- El parcial es en los laboratorios, usando las computadoras de los laboratorios o sus propias computadoras, siempre que sean soportadas por el sistema de parciales detallado más adelante.
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.
- Vamos a usar herramientas de detección de plagio para asegurarnos de que su entrega sea original. La entrega debe ser original para estar aprobada. El plagio en instancias de evaluación incurre sanciones que pueden incluir la suspensión académica.

Respecto al sistema de parciales:

- Deberán conectarse al sistema de parciales de la materia, el cual limitará su acceso a internet para que únicamente se puedan conectar a GitHub. Se debe **mantener la conexión** a este sistema hasta entregar el examen.
- Les proveeremos de un script que deberá ejecutarse al comienzo del parcial y no detenerse hasta haber entregado. Con él nos aseguramos que permanezcan conectados al sistema.
- Para aprobar el parcial deben implementar 2 de los 3 puntos del enunciado y que corran con éxito los tests funcionales, de abi y de memoria (valgrind).

Régimen de Aprobación

En este parcial se evalúa el manejo de los mecanismos de: - Excepciones, interrupciones de hardware y syscalls mediante Interrupciones al procesador - Manejo de memoria mediante Segmentación y Paginación - Permisos y cambios de nivel de privilegio - Administración del tiempo de procesamiento y del espacio de memoria para la ejecución de tareas.

Para la Arquitectura Intel x86. La entrega debe demostrar, en la medida exigida por el enunciado, comprensión clara de la mayoría estos mecanismos, e incomprensión de ninguno.

El parcial se aprueba con un puntaje numérico de **60 o más**, y califica para régimen de promoción con un puntaje numérico de **80 o más**.

NOTA: Lea el enunciado del parcial hasta el final antes de comenzar a resolverlo.

Régimen de Promoción (solo para quienes estén rindiendo por primera vez con ausencia justificada a la primer fecha)

Quien reciba un puntaje de **80 o más** en este parcial y tenga una calificación suficiente para promoción en el primer parcial podrá acceder al régimen de promoción. Esto es válido únicamente para la primer instancia de los parciales y no para sus recuperatorios.

Modalidad de Entrega

> [!WARNING] Se estará circulando por el aula una planilla escrita donde **deben** anotarse para que tengamos constancia de que su presencia en los laboratorios al momento del parcial. De no anotarse no podremos considerar su entrega como válida. La planilla, luego de circular, se encontrará en los escritorios al frente de cada laboratorio. **No se vayan sin anotarse!**

Deberán crear una nueva branch en este repositorio y allí desarrollar su solución en el archivo `Resolucion.md`, en formato Markdown (Acá tienen un machete de cómo dar formato en markdown). Es importante que no utilicen otro formato ya que dificultaría la corrección.

Si su solución utiliza código modificado del Trabajo Práctico, es importante que lo incluyan además de explicar en palabras las modificaciones (pueden copiar y pegar cualquier cosa de sus repositorios de TP). También pueden incluir imágenes si así lo desean. De tener dudas sobre qué funciones se pueden o no asumir implementadas, preguntar a los docentes durante el examen.

Una vez finalizado, completarán la entrega **creando un Pull Request e incluyendo a ayoc-bot como reviewer. Deben ver un mensaje de "Recibido!" de ayoc-bot para confirmar la entrega.**

Contexto

Partiendo de un sistema como el del TP, nos piden que lo extendamos para poder controlar el proceso de producción de monitores en una fábrica automatizada. En términos generales, las máquinas de la fábrica consumen unos recursos y producen otros. Por ejemplo, la máquina que produce el monitor terminado requiere: el panel led, la estructura, la carcasa, el chip controlador, etc. Todos estos recursos que necesita son producidos por otras máquinas, que a su vez pueden requerir recursos producidos por otras.

Para administrar este flujo de recursos, cada máquina estará controlada por una tarea que correrá sobre nuestro sistema. Las tareas en sí serán desarrolladas por otro equipo con el que acordamos algunas cuestiones: - El kernel proveerá una **syscall** `solicitar` que permita a la tarea llamadora solicitar un recurso (informando el tipo de recurso que busca). **No se devolverá el control** a la tarea solicitante hasta que el recurso esté listo. - El kernel proveerá una **syscall** `recurso_listo` que permita a la tarea llamadora avisar que su recurso está listo. La misma habrá dejado información en los 4KB a partir de la dirección virtual `0x0AAAA000` que **deberá ser copiada** a los 4KB a partir de la dirección virtual `0x0BBBB000` de la tarea que pidió el recurso (las direcciones podrían no estar mapeadas, en cuyo caso habría que **limpiarlas antes de acceder**). Una vez informado que el recurso está listo y copiada la información, la tarea llamadora debe ser desalojada y restaurada por completo a su estado inicial. - El kernel proveerá un mecanismo de arranque manual mediante una **interrupción externa** producida al presionar un botón, a la que le corresponde la entrada 41 de la IDT. Su efecto será iniciar la producción del `recurso_t` indicado en la dirección `0xFAFAFA` del espacio del kernel, bajo la misma lógica que `solicitar`.

Aclaraciones: - Toda tarea es productora y potencial solicitante. - Para todo recurso existe una tarea que lo produzca. - La fábrica está dispuesta de tal forma que no existen casos en los que se forman cadenas infinitas de dependencias (e.g. productora de A solicita recurso B que solicita recurso C que solicita recurso A). - Pueden asumir que las tareas ya fueron inicializadas.

Ejercicio 1 (50 puntos):

Detallar todas las modificaciones necesarias para que el kernel provea las sycalls `solicitar` y `recurso_listo`. Esto incluye pero **no se limita** a: - Establecer el mecanismo por el cual las tareas pasarán parámetros a las syscalls. - Modificar las estructuras necesarias para que las syscalls estén disponibles. - Establecer el mecanismo por el cual la tarea solicitante no retoma su ejecución hasta tener listo el recurso pedido. (Tener presente que durante la ejecución de una syscall, las interrupciones están deshabilitadas). - Establecer el mecanismo por el cual se copia la información de la tarea productora a la tarea solicitante.

Cuentan con las siguientes **funciones ya implementadas**, para este ejercicio no hace falta que consideren cómo están implementadas. Su uso es alentado pero no es obligatorio:

- ```
  task_id_t hay_tarea_disponible_para_recurso(recurso_t recurso)
  ```

  Dado un tipo de recurso, retorna el id de la tarea disponible para producir el recurso si es que hay, 0 si todas las tareas que producen el recurso indicado están ocupadas.

- ```
  task_id_t para_quien_produce(task_id_t id_tarea)
  ```

  Dado un id de tarea productora, devuelve el id de la tarea que solicitó la producción. Si la producción se solicitó mediante el arranque manual, devuelve 0. Si id_tarea corresponde a una tarea que no está produciendo, el comportamiento de la función es indefinido.

- ```
  void restaurar_tarea(task_id_t id_tarea)
  ```
  ```

Dado un id de tarea, realiza las modificaciones necesarias para que la próxima vez que sea ejecutada empiece desde el comienzo y sin arrastrar cambios de su ejecución anterior.

- `task_id_t hay_consumidora_esperando(recurso_t recurso)`

  Dado un tipo de recurso, retorna el id de una tarea que esté esperando que se disponibilice una productora del mismo. De no haber, retorna 0.

Nota: `task_id_t` es un renombre de `int8_t`.

Ejercicio 2 (25 puntos):

Implementar `restaurar_tarea`. La tarea debe quedar en condiciones para que cuando retome su ejecución, lo haga como si no se hubiese ejecutado antes. Pueden asumir que en su ejecución anterior no utilizó páginas nuevas aparte de `0x0AAAA000` y `0x0BBBB000`.

Considerar con detenimiento y detallar en qué punto de la ejecución del sistema se ejecuta esta función.

Ejercicio 3 (15 puntos):

Detallar todas las modificaciones necesarias para que el kernel provea el arranque manual mediante interrupción externa.

Se pueden asumir implementadas las funciones dadas en el ejercicio 1.

Ejercicio 4 (10 puntos):

Suponiendo que conocen de antemano qué recurso produce cada tarea, asumiendo que hay a lo sumo 10 tareas, y que cada una produce un único recurso, pero puede haber más de una que lo produzca. Considerar: - ¿Cómo modificarían el sistema para almacenar qué recurso produce cada tarea? - ¿Cómo modificarían el sistema para llevar registro de qué tareas están esperando a que se liberen productoras de su recurso solicitado?

De las funciones dadas, implementar aquellas que se hayan utilizado en la solución de los ejercicios anteriores, considerando sus respuestas a los subitems previos.

A tener en cuenta para la entrega

- Se parte de un sistema igual al del TP-SP terminado, sin ninguna funcionalidad adicional.
- Indicar todas las estructuras de sistema que deben ser modificadas para implementar las soluciones.
- Está permitido utilizar las funciones desarrolladas en las guías y TPs, siempre y cuando no utilicen recursos con los que el sistema no cuenta.
- Es necesario que se incluya una **explicación con sus palabras** de la idea general de las soluciones.
- Es necesario explicitar todas las asunciones que hagan sobre el sistema.
- Es necesaria la entrega de **código que implemente las soluciones**.
- `zero_page()` modifica páginas basándose en una **dirección virtual** (no física!)

Resolución segundo parcial - Arquitectura y Organización del Computador - Felipe Michalski

Nuevas definiciones

```
typedef uint32_t task_id_t;

typedef int32_t resource_t;
```

task__id__t es un alias para uint32__t__.

resource__t es un alias para uint32__t__ que representa su id

Actualización del scheduler

Para poder buscar una tarea que pueda producir un recurso o encontrar qué tarea realizó una solicitud, hago la siguiente modificación:

```
typedef struct {
  int16_t selector;
  task_state_t state;
  resource_t recurso_producido; // nuevo atributo
  resource_t recurso_pedido; // nuevo atributo
  tipo_e tipo; // nuevo atributo
} sched_entry_t;
```

Si el atributo *recurso_pedido* está en 0 significa que la tarea asociada no tiene ninguna solicitud en curso.

Sobre el atributo **tipo** volveré más adelante.

Además, voy a agregar dos estados al enumerado de estados del scheduler.

El estado **TASK_WAITING_REQUEST** representa a una tarea que está esperando que otra solicite el recurso que esta produce y su fin es permitirle al scheduler encontrar tareas que estén listas para producir recursos solicitados.

El estado **TASK_WAITING_RESOURCE** representa a una tarea que solicitó un recurso que todavía no está listo. Esto alcanza para bloquear las tareas que solicitaron recursos mientras se producen porque la función *sched_next_task* (que retorna el id de la siguiente tarea a ejecutar) solo devuelve ids de tareas en estado **TASK_RUNNABLE**.

También tengo que modificar la función sched_add_task para que toda tarea perteneciente a mi sistema se cree con estado **TASK_WAITING_REQUEST** (similar al *TASK_PAUSED* pero dedicado a esta funcionalidad) y el id del recurso que produce. Todos los llamados previos a esta tarea setearían su parámetro *is_resource_task* en *FALSE* *recurso_producido* en 0.

```
typedef enum {
  TASK_SLOT_FREE,
  TASK_RUNNABLE,
  TASK_PAUSED,
  TASK_WAITING_REQUEST, // nuevo estado
  TASK_WAITING_RESOURCE // nuevo estado
} task_state_t;

int8_t sched_add_task(uint16_t selector, bool is_resource_task, recurso_t resource_id, tipo_e tipo) {
  kassert(selector != 0, "No se puede agregar el selector nulo");

  // Se busca el primer slot libre para agregar la tarea
  for (int8_t i = 0; i < MAX_TASKS; i++) {
    if (sched_tasks[i].state == TASK_SLOT_FREE) {
      if(is_resource_task) {
          sched_tasks[i] = (sched_entry_t) {
            .selector = selector,
            .state = TASK_WAITING_REQUEST,
            .recurso_producido = resource_id,
            .recurso_pedido = 0,
            .tipo = tipo
          };
          return i;
      } else {
          sched_tasks[i] = (sched_entry_t) {
            .selector = selector,
            .state = TASK_PAUSED,
            .recurso_producido = resource_id,
            .recurso_pedido = 0
            .tipo = tipo
          };
```

```
            return i;
        }
      }
    }
  kassert(false, "No task slots available");
}
```

Las tareas que no pertenecen a nuestro sistema de recursos pueden tener el atributo *recurso_pedido* en 0 dado que nunca tendrán un estado asociado a nuestro sistema (*TASK_WAITING_REQUEST*, *TASK_WAITING_RESOURCE*) por ende serán omitidas.

Los atributos *recurso_pedido* y *recurso_producido* están pensados para las implementaciones de **hay_tarea_disponible_para_recurso, para_quien_produce y hay_consumidora_esperando** en conjunto con los nuevos estados.

Este código debería estar ubicado en el archivo *sched.c*

NOTA: a lo largo de esta solución usaré la variable global de kernel *sched_tasks* asumiendo que el tamaño de la lista (la variable *MAX_TASKS*) se aumenta en una cantidad igual a las tareas que solicitan y producen recursos.

Codigo para ejecutar las syscall

Defino los siguiente números de interrupciones para mis syscalls: - solicitar -> 90 - recurso_listo -> 91

Estos códigos de interrupción pertenecen al rango de interrupciones de software.

Ahora bien, en primer lugar en la IDT agrego una nueva entrada para cada nueva interrupción de la siguiente forma:

```
  IDT_ENTRY0(41);
  IDT_ENTRY3(90);
  IDT_ENTRY3(91);
```

La primera entrada corresponde a la interrupción por **arranque manual** y tiene DPL de kernel (ENTRY0) porque es una interrupción de hardware, no es deseado que se pueda invocar como una syscall.

Las otras dos entradas tienen DPL de usuario (ENTRY3) porque como son syscalls, cuando sean llamadas por una tarea, el CPL de esta va a ser de nivel 3 (usuario) y, al compararlo con el DPL de la entrada, si no coinciden no permitirá su ejecución.

Este código debería estar ubicado en el archivo *idt.c*.

Rutinas que disparan las interrupciones (las que puede invocar el usuario)

```
LS_INLINE void solicitar(resource_t recurso) {
  __asm__ volatile (
      "mov %0, %%eax\n\t"
      "int $90\n\t"       // llamo a la interrupcion 90
      : "r"(recurso)   // entrada: recurso
      : "eax"             // pisa eax
  );
}

LS_INLINE void recurso_listo(resource_t recurso) {
  __asm__ volatile (
      "mov %0, %%eax\n\t"
      "int $91\n\t"       // llamo a la interrupcion 90
      : "r"(recurso)   // entrada: recurso
      : "eax"             // pisa eax
  );
}
```

La syscall **solicitar** devuelve void dado que siempre se escribe en la misma dirección virtual, por ende se sabe dónde ir a buscar el recurso solicitado.

Definicion de las nuevas interrupciones para las syscalls

Dada una de estas rutinas _isrXX maneja la interrupción asociada a la syscall donde XX corresponde al número de interrupción asociado.

```
extern solicitar
extern recurso_listo

global _isr41
_isr41:
  pushad

  mov eax, 0xFAFAFA
  push eax // paso el parámetro recurso recibido en esa direccion por enunciado
  call iniciar_produccion
  add esp, 4 // restauro la pila

  popad
  iret

global _isr90
_isr90:
  pushad

  push eax // paso el parámetro recurso
  call solicitar
  add esp, 4 // restauro la pila

  call sched_next_task
  mov word [sched_task_selector], ax
  jmp far [sched_task_offset]

  popad
  iret

global _isr91
_isr91:
  pushad

  push eax // paso el parámetro recurso
  call recurso_listo
  add esp, 4 // restauro la pila

  call sched_next_task
  mov word [sched_task_selector], ax
  jmp far [sched_task_offset]

  popad
  iret
```

Como se puede ver, en las dos rutinas de syscalls, luego de llamar al handler se pide el selector de la siguiente tarea y se ejecuta un jmp far. Esto es porque, dentro del handler, o bien para que la tarea quede bloqueada hasta que el recurso solicitado esté listo o bien porque ya terminó de producir lo que le correspondía, se actualiza el estado de esta a uno distinto de **TASK_RUNNABLE**. Esto permite que, luego de salir del handler, al llamar a la funcion *sched_next_task* el identificador de tarea devuelto siempre sea distinto al de la tarea que invocó la syscall por la política del scheduler. En la rutina de interrupción de hardware no es necesario porque, antes de ocurrir dicha interrupción, ya había otra tarea corriendo y se desea que el sistema continúe su ejecución normalmente.

Handlers de las syscalls

En este apartado defino cómo se va a manejar cada syscall una vez que está en la interrupción y algunas funciones auxiliares que necesito para esto.

```c
bool mmu_is_mapped(uint32_t cr3, vaddr_t virt) {
  pd_entry_t* pd = (pd_entry_t*) CR3_TO_PAGE_DIR(cr3);
  uint32_t pd_index = (uint32_t) VIRT_PAGE_DIR(virt);

  if(!(pd[pd_index].attrs & MMU_P)){ //chequeo si el indice en el directorio esta prensente
    return false;
  }

  pt_entry_t* pt = (pt_entry_t*) MMU_ENTRY_PADDR(pd[pd_index].pt);
  uint32_t pt_index = (uint32_t) VIRT_PAGE_TABLE(virt);

  if (pt[pt_index].attrs & MMU_P){
    return true;
  } else {
    return false;
  }
}
```

El objetivo de esta función es determinar si una dirección virtual está mapeada dentro de un page directory dado. Me permite saber si tengo que realizar mapeos antes de copiar la información de un recurso al ejecutarse la syscall de **recurso_listo** (ya que la función mmu_map_page no valida si una página está mapeada al ser invocada).

Este código debería estar ubicado en el archivo *mmu.c*

Luego defino dos funciones más:

```c
uint32_t task_selector_to_CR3(uint16_t selector) {
  uint16_t index = selector >> 3; // Saco los atributos
  gdt_entry_t* descriptor = &gdt[index]; // Indexamos en la gdt
  tss_t* tss = (tss_t*)((descriptor->base_15_0) | (descriptor->base_23_16 << 16) |
↪  (descriptor->base_31_24 << 24)); // armo la dirección de la tss
  return tss->cr3; // obtengo su CR3
}

paddr_t virt_to_phy(uint32_t cr3, vaddr_t virt) {
  uint32_t pd_index = VIRT_PAGE_DIR(virt);
  uint32_t pt_index = VIRT_PAGE_TABLE(virt);
  pd_entry_t* pd = (pd_entry_t*) CR3_TO_PAGE_DIR(cr3);
  pt_entry_t* pt = pd[pd_index].pt << 12;
  return (paddr_t) (pt[pt_index].page << 12);
}
```

Estas funciones me sirven para obtener las direcciones físicas a las que están mapeadas direcciones virtuales de diferentes tareas (ya que recibe el CR3 por parámetro). Las voy a usar para copiar la información escrita por la tarea productora a la región destinada a la tarea solicitante.

Las macros **VIRT_PAGE_DIR** y **CR3_TO_PAGE_DIR** ya están definidas dentro de nuestro sistema.

La primera función podría estar ubicada en el archivo *tss.c* y la segunda en *mmu.c*

Implementación de los handlers

```c
void iniciar_produccion(resource_t recurso) {
  task_id_t tarea_productora = hay_tarea_disponible_para_recurso(recurso);
  if(tarea_productora != 0) {
    sched_tasks[tarea_productora].state = TASKS_RUNNABLE;
  }
}
```

Esta implementación asume que **el arranque manual ocurre una sola vez** para iniciar el sistema y nunca más, por eso es que no actualiza su estado propio ni su recurso pedido (de hecho, si se quisiera hacer lo mismo que

solicitar, el current_task devolvería un valor que no es útil porque es una interrupción de hardware y puede ocurrir mientras se procesa cualquier tarea, quedando esta asociada a ese valor).

```
void solicitar(resource_t recurso) {
  task_id_t tarea_productora = hay_tarea_disponible_para_recurso(recurso);
  if(tarea_productora != 0) {
    sched_tasks[tarea_productora].state = TASKS_RUNNABLE;
  }
  sched_tasks[current_task].state = TASKS_WAITING_RESOURCE;
  sched_tasks[current_task].recurso_pedido = recurso;
}
```

La idea de este handler es avisarle al sistema que la tarea actual solicita un recurso. Para ello, cambia su estado a "esperando recurso" y actualiza el identificador del recurso que desea. En caso de que haya una tarea que puede producirlo y esté libre la marca como TASK_RUNNABLE para que empiece la producción.

```
#define PRODUCER_START_VIRT 0x0AAAA000
#define REQUESTER_START_VIRT 0x0BBBB000

void recurso_listo(resource_t recurso) {
  task_id_t tarea_consumidora = para_quien_produce(current_task);

  if(tarea_consumidora != 0) {
    uint32_t cr3_productora = rcr3();
    uint32_t cr3_consumidora = task_selector_to_CR3(sched_tasks[tarea_consumidora].selector);

    // si la tarea consumidora no tiene memoria mapeada para recibir la informacion del recurso la
     ↪  mapeo
    if(!mmu_is_mapped(cr3_consumidora, REQUESTER_START_VIRT)) {
      paddr_t phy = mmu_next_free_user_page();
      mmu_map_page(cr3_consumidora, REQUESTER_START_VIRT, phy, MMU_U | MMU_W);
      zero_page(REQUESTER_START_VIRT);
    }

    paddr_t phy_productora = virt_to_phy(cr3_productora, PRODUCER_START_VIRT)
    paddr_t phy_solicitante = virt_to_phy(cr3_consumidora, REQUESTER_START_VIRT);

    copy_page(phy_solicitante, phy_productora);

    sched_tasks[tarea_consumidora].state = TASK_RUNNABLE;
    sched_tasks[tarea_consumidora].recurso_pedido = 0;
  }

  restaurar_tarea(current_task);

  task_id_t consumidora_en_espera = hay_consumidora_esperando(recurso);
  if(consumidora_en_espera == 0) {
    sched_tasks[current_task].state = TASKS_WAITING_REQUEST;
  }
}
```

En esta implementación asumo que la tarea llamadora ya escribió **en su región de memoria asignada** la información del recurso. También asumo que la dirección virtual en la que arranca su región de escritura siempre está mapeada (escribe la información e invoca a esta syscall, no desmapea en el medio).

Como el enunciado dice que la región donde se deja la información es de 4KB y este es el tamaño de una página, alcanza con hacer una sola copia.

Este copiado de datos se puede hacer porque, al estar dentro de una syscall, se ejecuta con privilegio de supervisor (kernel). Por este mismo motivo, variables como *sched_tasks* o *current_tasks* definidas globalmente dentro de la memoria del kernel (en el archivo *sched.c*) o funciones como *mmu_map_page*, *mmu_is_mapped*, *mmu_next_free_user_page* son accesibles.

Si la producción esta causada por el arranque manual entonces no hay ningún dato que escribir puesto que es el "inicio" de la producción y nadie está esperando la información de ese recurso.

Finalmente, luego de hacer (o no) el copiado, restaura la tarea que es lo que se pide. Si hay consumidores en espera se deja la tarea en TASK_RUNNABLE (estado actual). Como ya fue restaurada, cuando se vuelva a ejecutar arrancará de cero y producirá el recurso nuevamente. Si no hay consumidores, la marca como "esperando a que haya consumidores de vuelta".

NOTA: Todas las funciones usadas en estas implementaciones que no están definidas dentro de la solución ya existen en el sistema sobre el cual se apoya.

Implementación de restaurar_tarea

Esta función es la encargadad de realizar las modificaciones necesarias para que la próxima vez que una tarea ejecutada empiece desde el comienzo y sin arrastrar cambios de su ejecución anterior.

Antes de pasar a su implementación, redefino los tipos de mis tareas en el archivo *tasks.c*

```
/**
 * Tipos de tareas soportados por el sistema
 */
typedef enum {
  TASK_A = 0,
  TASK_B = 1,
  ...,
  TASK_N = X,
} tipo_e;

/**
 * Array que nos permite mapear un tipo de tarea a la dirección física de su
 * código.
 */
static paddr_t task_code_start[2] = {
    [TASK_A] = TASK_A_CODE_START,
    [TASK_B] = TASK_B_CODE_START,
    ...,
    [TASK_N] = TASK_N_CODE_START,
};
```

Para cada nuevo recurso yo voy a crear un tipo nuevo de tarea asociado a una direccion donde arranca su codigo. Teniendo esto en cuenta, paso a definir *restaurar_tarea* como:

```
void restaurar_tarea(task_id_t id_tarea) {
  uint32_t cr3 = task_selector_to_CR3(sched_tasks[id_tarea].selector);
  zero_page(PRODUCER_START_VIRT);
  zero_page(REQUESTER_START_VIRT);
  paddr_t phy1 = mmu_unmap_page(cr3, (vaddr_t) PRODUCER_START_VIRT);
  paddr_t phy2 = mmu_unmap_page(cr3, (vaddr_t) REQUESTER_START_VIRT);

  tipo_e tipo_tarea = sched_tasks[id_tarea].tipo // para esto defino este atributo

  uint16_t gdt_index = sched_tasks[id_tarea].selector >> 3;
  gdt_entry_t* descriptor = &gdt[gdt_index];
  tss_t* tss = (tss_t*)((descriptor->base_15_0) | (descriptor->base_23_16 << 16) |
↪  (descriptor->base_31_24 << 24));
  tss->eip = task_code_start[tipo_tarea];
  tss->esp = tss.ebp;
}
```

La idea es desmapear las paginas que fueron utilizadas para leer/escribir información de recursos que produjo o solicitó, limpiarlas y luego reiniciar su tss a los valores iniciales (el instruction pointer al inicio de la tarea y el tope del stack nuevamente al inicio).

Esta función, como fue mencionado e implementado anteriormente, está pensada para ser usada luego de que una tarea ejecuta la syscall *recurso_listo* y se copia la información del recurso que produjo a la región de memoria de la tarea solicitante.

NOTA: No es necesario revisar si las páginas que se desea desmapear fueron o no mapeadas porque ya lo hace la implementación de la función *mmu_unmap_page*.

Page fault

Para esta solución, hay un caso en particular de *page fault* que quiero manejar de forma distinta; cuando una tarea productora está escribiendo **por primera vez** información del recurso que produjo en su región de memoria asignada (en la creación de la tarea no se mapean estás direcciones). Cuando esto ocurre, se debe realizar el mapeo correspondiente.

No es necesario modificar la rutina que atiende la interrupción de page fault (la *_isr14*) pero sí la función **page_fault_handler**, definida en el archivo *mmu.c*, a la que esta invoca. El resultado sería este:

```
bool page_fault_handler(vaddr_t virt) {
  uint32_t cr3 = rcr3();

  // esta implementación la mantengo como estaba en nuestro sistema inicialmente
  if(virt >= ON_DEMAND_MEM_START_VIRTUAL && virt <= ON_DEMAND_MEM_END_VIRTUAL){
    mmu_map_page(rcr3(), ON_DEMAND_MEM_START_VIRTUAL, ON_DEMAND_MEM_START_PHYSICAL, MMU_U | MMU_W);
    return 1;
  }

  if(virt >= PRODUCER_START_VIRT && virt <= PRODUCER_START_VIRT + PAGE_SIZE) {
    paddr_t phy = mmu_next_free_user_page();
    zero_page(phy);
    mmu_map_page(rcr3(), PRODUCER_START_VIRT, phy, MMU_U | MMU_W);
    return 1; // pudo atender el page fault, no falla
  }
  return 0;
}
```

De esta forma, la primera vez que quiera escribir allí recibirá un page fault, hará el mapeo y permitirá la escritura y las siguientes podrán hacerlo sin problema.

Respuestas ejercicio 4

¿Cómo modificarían el sistema para almacenar qué recurso produce cada tarea?

Mi sistema ya lo hace con el atributo **recurso_producido** definido en cada una de las tareas que se guardan en la lista de tareas del scheduler.

¿Cómo modificarían el sistema para llevar registro de qué tareas están esperando a que se liberen productoras de su recurso solicitado?

Podría agregar un estado más a la tarea y hacer una distinción entre: - TASK_WAITING_RESOURCE_IN_PROCESS - TASK_WAITING_RESOURCE_UNASSIGNED

De esta forma, podría distinguir cuáles solicitudes ya están siendo procesadas y cuales no.

A continuación, implemento las 3 funciones utilizadas anteriormente que faltan por definir:

```
task_id_t hay_consumidora_esperando(recurso_t recurso) {
  for(int i = 0; i < MAX_TASKS; i++) {
    if(sched_tasks[i].recurso_pedido == recurso && sched_tasks[i].state ==
    ↪  TASK_WAITING_RESOURCE_UNASSIGNED && i != current_task) {
       return i;
    }
  }
```

```
  return 0;
}

task_id_t para_quien_produce(task_id_t id_tarea) {
  recurso_t recurso = sched_tasks[id_tarea].recurso_producido;
  for(int i = 0; i < MAX_TASKS; i++) {
    if(sched_tasks[i].recurso_pedido == recurso && sched_tasks[i].state ==
     ↪  TASK_WAITING_RESOURCE_IN_PROCESS && i != current_task) {
        return i;
    }
  }
  return 0;
}

task_id_t hay_tarea_disponible_para_recurso(recurso_t recurso) {
  for(int i = 0; i < MAX_TASKS; i++) {
    if(sched_tasks[i].recurso_pedido == recurso && sched_tasks[i].state == TASK_WAITING_REQUEST && i
     ↪  != current_task) {
        return i;
    }
  }
  return 0;
}
```

Correcciones

Hola @felimichalski! Felicitaciones! Tu segundo parcial está aprobado con 95.

Ej1: 50/50

Ej2: 20/25

Ej3: 15/15

Ej4: 10/10

Ej1: + Bien.

Ej2: + Está bien que estés intentando restaurar el `eip`, `ebp` y `esp`. El problema es que estás intentando hacerlo modificando la `tss` de la tarea corriendo.

Cuando suceda un `context switch`, el contenido de la `tss` se sobreescribe automáticamente con la situación en el momento del `context switch`, por lo que los registros que restauraste son pisados nuevamente.

Si quisieras restaurar por `tss`, debería ser el scheduler o la siguiente tarea quien lo haga, para que el reinicio no sea pisado por el `context switch`. Una tarea nunca puede modificar su propia `tss`.

Otra manera un poco más simple de hacerlo sería pisar los valores que se pushean automáticamente durante una interrupción, que son, entre otros, el EIP, ESP y EBP. Luego de un IRET, los valores dentro de la pila pisan a los registros y la tarea queda reiniciada.

+ El desmapeo de las páginas está bien.

Ej3: + Bien.

Ej4: + Bien.